

Security Audit

FROGY by dTelecom (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The FROGY App (dTelecom) team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

dTelecom has launched FROGY, a livestreaming application that enables users to host streams and earn through gifts, tips, and subscriptions from viewers. Streamers receive 90% of the revenue, while the platform retains 10%. Deposits and withdrawals are instant and fee-free. The application is powered by smart contracts deployed on Solana. In parallel, dTelecom is building a decentralized real-time communication (dRTC) platform that combines voice, video, chat, and AI. Its goal is to enable developers to embed communication features (voice calls, video conferencing, interactive livestreaming, etc.) into their applications at significantly lower cost, while giving participants ownership and control of their data. Users can also earn by contributing bandwidth (running nodes) or through affiliate programs.

The project is powered by a native token (DTEL), is open source, ensures low latency and high resilience through its decentralized network architecture, and integrates AI enhancements such as real-time translation, voice agents, noise suppression, and more.

Project Name: FROGY App (dTelecom)

Project Type: dApp

Compiler Version: N/A

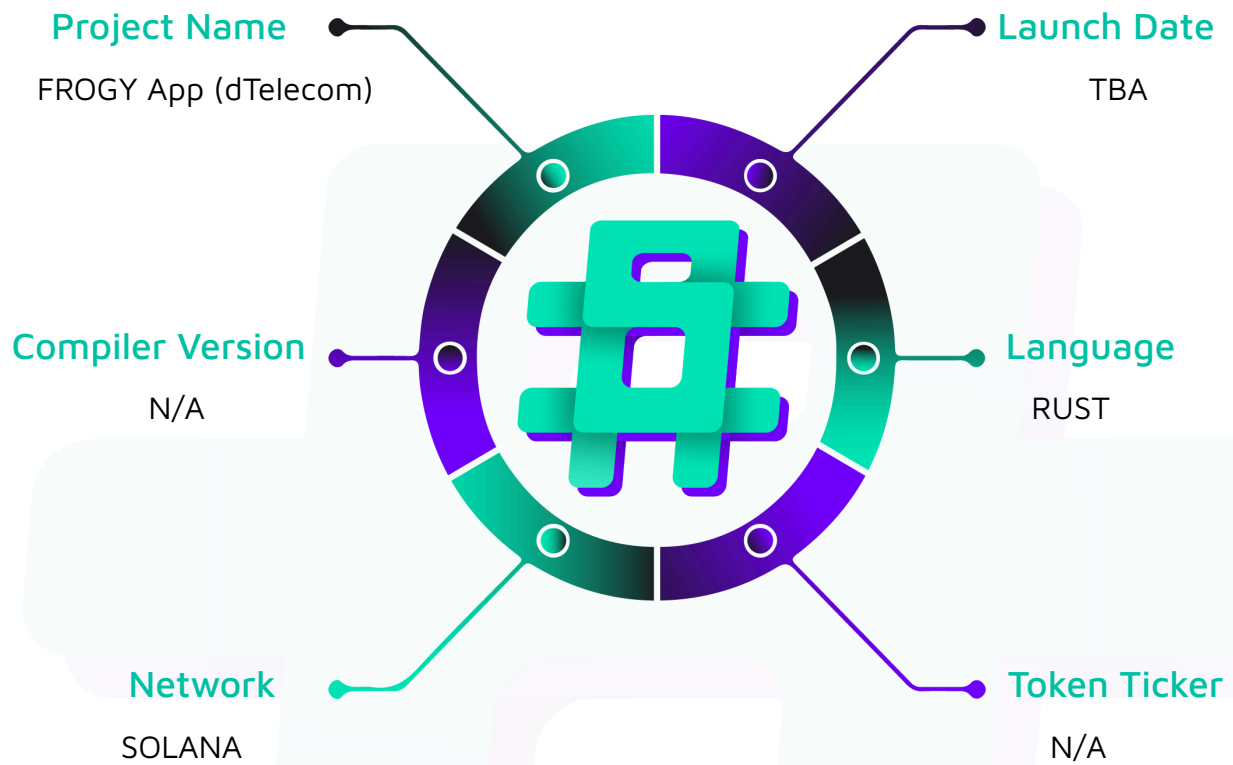
Website: <https://froggy.com>, <https://dtelecom.org>

Logo:

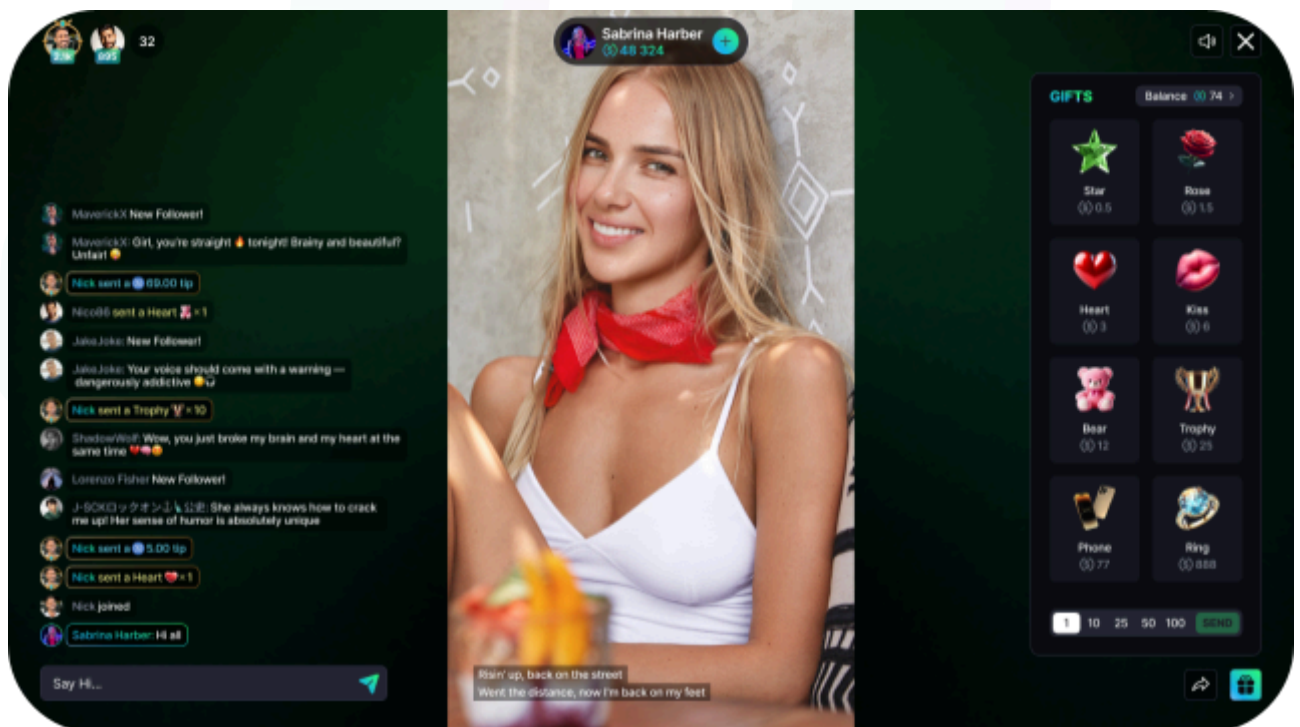
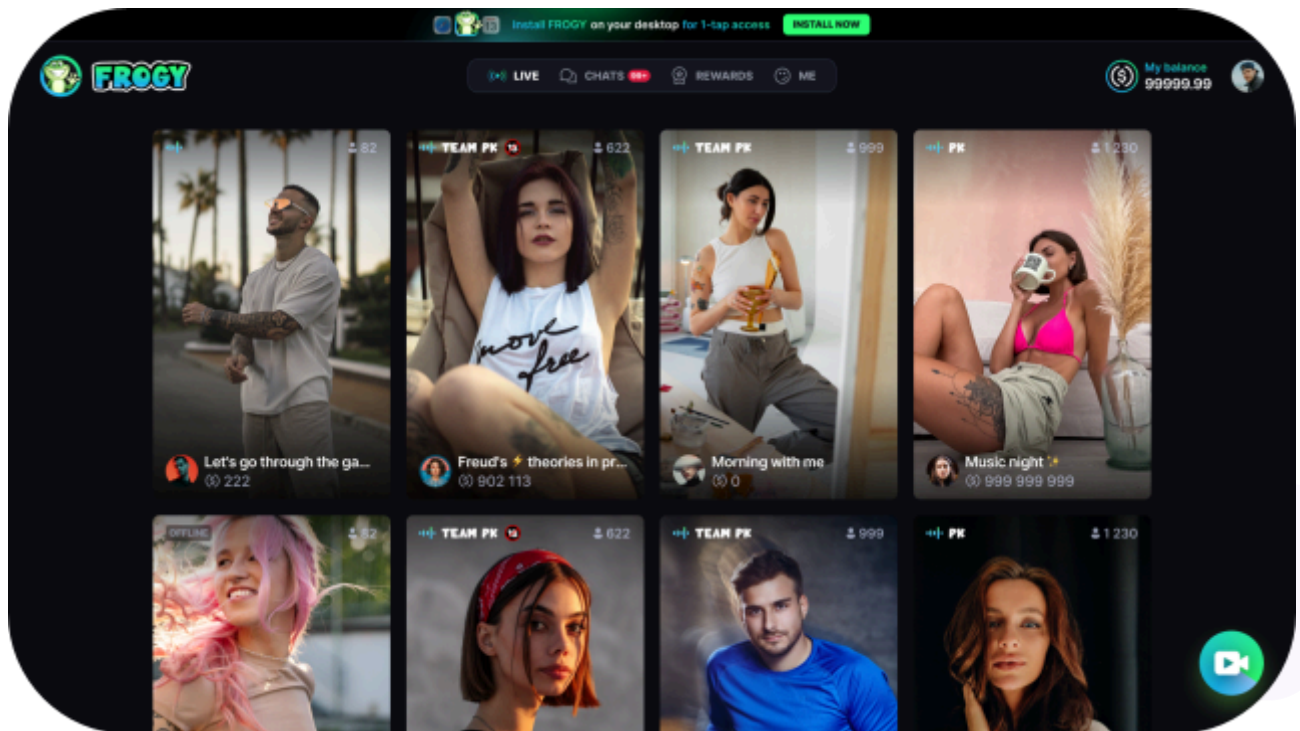


#hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the FROGY App (dTelecom) project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	FROGY App (dTelecom) Smart Contracts
Platform	Solana / Rust
Audit Date	September, 2025
Contract 1	Lib.rs
Audited GitHub Commit Hash	e8d48395b1799e84957486560d355373489d34b2
Fix Review GitHub Commit Hash	97fadd19a65770c6b5f2d8034e9bad7e793b16a3

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
fast-transfers/lib.rs - The Program Aims to make transfers among users in the same system to be faster by introducing internal transfer mechanism where users can deposit their USDC and can transfer those tokens to other users in the same system internally with internal accounting rather than making a token transfer everytime	Program achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the FROGY App (dTelecom) project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the FROGY App (dTelecom) project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] lib.rs#set_config,initialize_balance - Missing Signer Check

Description

The `SetConfig` and `initialize_balance` structs do not validate that the transaction signer is an authorized admin. Any user can call these functions and set themselves as the admin, gaining complete control over the protocol.

Vulnerability Details

These structs allow any user to initialize a balance and config account for any arbitrary owner without requiring the owner to sign the transaction. The owner parameter is passed as an instruction parameter but is never validated against any signer.

```
#[derive(Accounts)]
#[instruction(admin: Pubkey)]
pub struct SetConfig<'info> {
    #[account(init, payer = payer, space = Config::LEN, seeds = [b"config"], bump)]
    pub config: Account<'info, Config>,
    #[account(init_if_needed, payer = payer, space = UserBalance::LEN, seeds =
[b"balance", admin.as_ref()], bump)]
    pub admin_balance: Account<'info, UserBalance>,
    #[account(mut)]
    pub payer: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

```
#[derive(Accounts)]

#[instruction(admin: Pubkey)]

pub struct SetConfig<'info> {

    #[account(init, payer = payer, space = Config::LEN, seeds = [b"config"], bump)]

    pub config: Account<'info, Config>,

    #[account(init_if_needed, payer = payer, space = UserBalance::LEN, seeds =
[b"balance", admin.as_ref()], bump)]

    pub admin_balance: Account<'info, UserBalance>,

    #[account(mut)]

    pub payer: Signer<'info>,

    pub system_program: Program<'info, System>,

}
```

Impact

An attacker can create balance accounts for arbitrary users without their consent. Moreover, unauthorized users can take over protocol administration by setting their own config, setting arbitrary fees, and draining user funds by receiving fees.

Recommendation

Implement access control and set the protocol owner at the time of initialization and make sure that only authorized users can call admin functions.

Status

Resolved

[H-02] lib.rs#set_config,initialize_balance - DoS by PDA Griefing Attack

Description

A PDA squatting attack occurs when an attacker creates an account at a predictable address before the legitimate program does. Since an account can only be initialized once, this can permanently prevent the program from using that address, leading to a denial of service. The `set_config` and `initialize_balance` functions in the fast-transfers program are vulnerable to this attack due to their use of predictable seeds for PDA creation.

Vulnerability Details

1. `set_config` Function: The config account PDA is derived from a static and predictable seed: `b"config"`

```
pub struct SetConfig<'info> {
    #[account(init, ..., seeds = [b"config"], bump)]
    pub config: Account<'info, Config>,
    ...
}
```

2. `initialize_balance` Function: The user_balance account PDA is derived from the seeds `[b"balance", owner.as_ref()]`

```
pub struct SetConfig<'info> {
    #[account(init, ..., seeds = [b"config"], bump)]
    pub config: Account<'info, Config>,
    ...
}
```

An attacker can compute these addresses off-chain and create an account there by sending it some lamports before the legitimate administrator runs the `set_config` instruction and `initializes_balance` for the first time. This will cause the init check to

fail, as the account already exists and is owned by the system program instead of the fast-transfers program.

Impact

If an attacker successfully squats these PDAs, the instructions can never be successfully executed. This would render the entire protocol non-functional permanently, as the core configuration can never be initialized.

Recommendation

Use unique or unpredictable seeds for the config PDA

Status

Resolved

Medium

[M-01] lib.rs#set_config - Unbounded Fee Setting

Description

An administrator can set an unlimited fee percentage, risking protocol insolvency and failed transactions. The `fee_bps` parameter lacks validation, allowing fees of 100% or more. This issue is worsened because any user can change their config due to a missing signer check.

Vulnerability Details

The `set_config` function accepts a `u16` integer for the `fee_bps` parameter. A `u16` can hold a value up to 65,535. As per the current fee calculation, 10,000 basis points (bps) equals 100%. The function directly assigns this value to the configuration without checking if it is within a reasonable range.

```
pub fn set_config(ctx: Context<SetConfig>, admin: Pubkey, fee_bps: u16) -> Result<()> {  
    let config = &mut ctx.accounts.config;  
  
    config.admin = admin;  
  
    config.fee_bps = fee_bps; // <-- No validation on fee_bps  
  
    Ok(()) }
```

Impact

Fee setting up to 100% will lead to permanent loss of funds

Recommendation

The `fee_bps` parameter must be strictly validated to ensure it does not exceed a reasonable maximum limit. A require check should be added at the beginning of the `set_config` function.

Status

Resolved

[M-02] lib.rs#transfer_with_referer - Self-Referral Allows Fee Evasion

Description

The `transfer_with_referer` function is designed to split the transaction fee between the protocol's admin and a referrer who brings in users. However, the function lacks a crucial validation check to ensure that the transaction sender (`from_user`) cannot also be the `referrer_user`. This oversight allows any user to specify themselves as the referrer for their own transaction, effectively reclaiming 50% of the fee they were supposed to pay.

Vulnerability Details

The vulnerability exists because the program does not enforce that the sender and the referrer must be different entities.

```
// programs/fast-transfers/src/lib.rs

#[derive(Accounts)]

#[instruction(to_user: Pubkey, referer_user: Pubkey)] // referer_user comes from user
input

pub struct TransferWithReferer<'info> {

    // ...

    #[account(init_if_needed, payer = payer, space = UserBalance::LEN, seeds = [b"balance",
    referer_user.as_ref()], bump)]

        pub referer_balance: Account<'info, UserBalance>,

        pub from_user: Signer<'info>, // The attacker signs here

        // ...

        // NO CHECK to ensure from_user.key() != referer_user

}
```

Impact

Malicious users can consistently pay only 50% of the intended transaction fee by referring themselves. This leads to a direct loss of revenue for the protocol's admin, who should be receiving the full fee in the absence of a legitimate, separate referrer.

Recommendation

Add a validation check to the `transfer_with_referer` instruction to ensure the sender and referrer are not the same person. This can be easily implemented with a `require!` macro at the beginning of the instruction handler.

Status

Resolved

Low

[L-01] lib.rs - Missing Account Closure Mechanism and Permanent Locking of Rent

Description

The `fast-transfers` program creates several types of accounts (`Config` , `UserBalance` , and the vault `token` account) but provides no mechanism to close them. On `Solana`, when an account is created, `SOL` is paid to make it rent-exempt. This `SOL` is intended to be recoverable by closing the account. The absence of a closure function means that the `SOL` paid for rent on every created account is permanently locked and cannot be reclaimed.

Vulnerability Details

1. The `SetConfig` and `InitializeBalance` instructions (as well as `Deposit` , `Transfer` , etc.) use the `#[account(init, payer = payer, ...)]` or `#[account(init_if_needed, payer = payer, ...)]` attribute. This correctly allocates space and makes the new account rent-exempt by debiting `SOL` from the payer . However, no corresponding instruction exists to reverse this process. There is no `close_balance` or `close_config` function that would allow the original payer to reclaim this `SOL`.
2. The main vault `token` account, which holds all user deposits, is a Program-Owned Account whose authority is the `vault_pda` .. The program lacks any function that would allow the protocol's administrator to sign via the `vault_pda` to close this token account. Therefore, the `SOL` used to make the vault rent-exempt is also permanently locked.

Impact

This issue causes a slow but irreversible loss of `SOL` for both the protocol's deployer (for the `Config` and vault accounts) and for every user who creates a `UserBalance` account. While the amount per account is small (typically `0.001-0.002 SOL`), this adds up over time and across thousands of users.

Recommendation

Implement a closure mechanism for all accounts that store state.

Status

Resolved

[L-02] lib.rs - Panic on Arithmetic Overflow Instead of Returning Custom Error

Description

Throughout the `transfer` and `transfer_with_referer` functions, all `checked_mul`, `checked_div`, `checked_add`, and `checked_sub` operations are immediately followed by `.unwrap()`. If any of these arithmetic operations were to fail (e.g., due to an overflow), the entire transaction would panic and fail with a generic, unhelpful error code.

Impact

While using `checked` math correctly prevents incorrect state changes (like a balance wrapping around), panicking is not user-friendly. It makes it difficult for off-chain clients and developers to debug the exact cause of a transaction failure.

Recommendation

Replace `.unwrap()` with `.ok_or(CustomError::MathOverflow)?`. This will cause the transaction to fail gracefully with a specific, descriptive error

Status

Resolved

QA

[Q-01] lib.rs - Missing Events

Description

The program executes several critical state changes and financial transactions, but fails to emit on-chain events.

Vulnerability Details

All the state-changing functions in the program do not emit any events

Recommendation

We recommend emitting events for all state-changing operations

Status

Acknowledged

[Q-02] lib.rs - Redundant and Unused Function Parameters

Description

Several functions take Pubkey parameters that are immediately ignored, as the actual addresses are derived and constrained within the `#[derive(Accounts)]` structs.

Vulnerability Details

1. `initialize_balance(ctx: Context<InitializeBalance>, _owner: Pubkey)` : The `_owner` parameter is unused. The owner's key is already used as a seed for the `user_balance` PDA.
2. `transfer(ctx: Context<Transfer>, _to_user: Pubkey, ...)` : The `_to_user` parameter is unused. The recipient's account is derived from the `to_user` pubkey provided in the `#[instruction]` macro.
3. `transfer_with_referer(ctx: Context<TransferWithReferer>, _to_user: Pubkey, _referer_user: Pubkey, ...)` : Both `_to_user` and `_referer_user` are unused for the same reason.

Recommendation

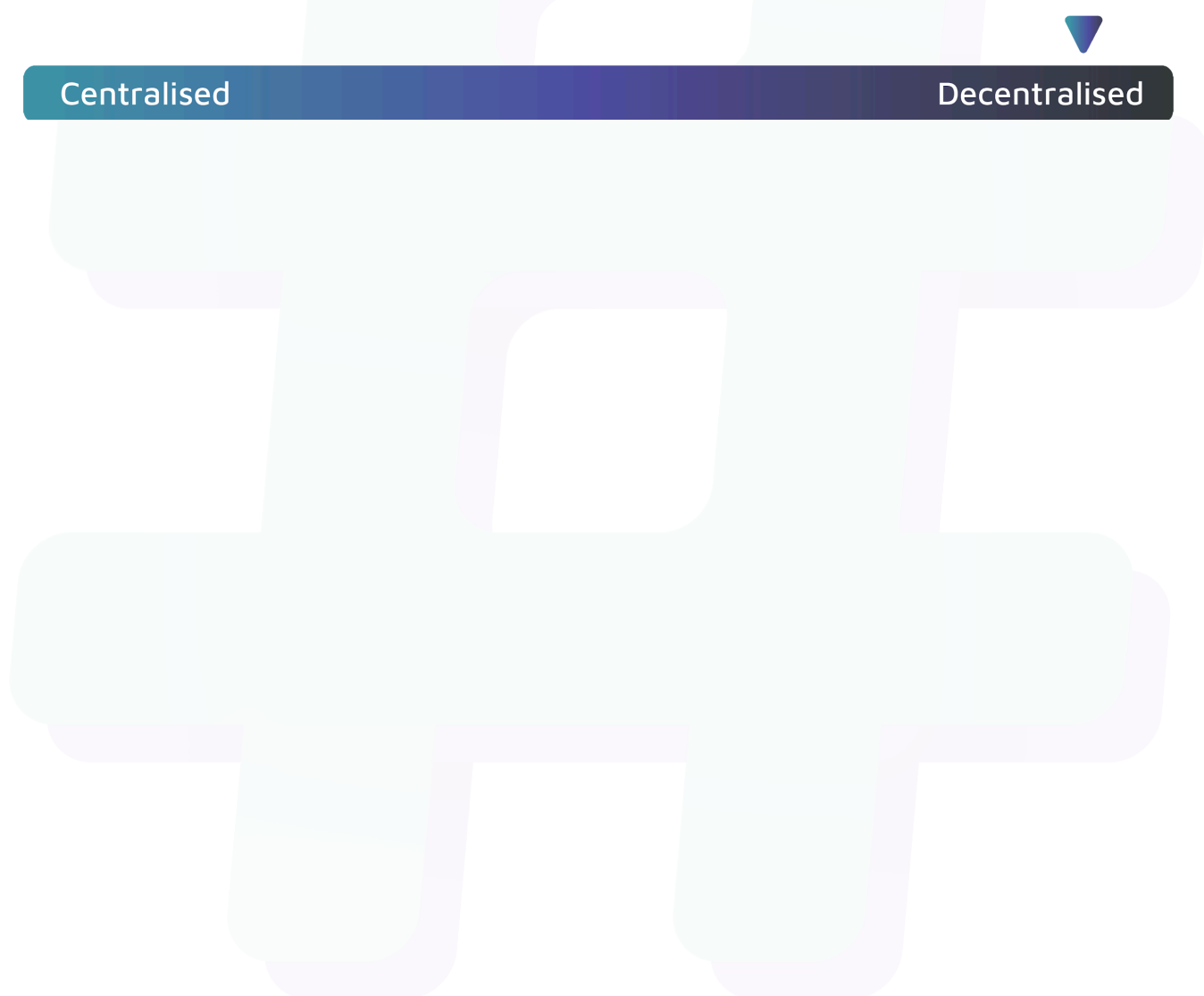
We recommend removing these unused parameters from the function signatures entirely.

Status

Resolved

Centralisation

The FROGY App (dTelecom) is built as a permissionless protocol—anyone can deposit USDC, transfer to another user, or withdraw without restriction. After the upgrade, the authority is revoked, and the protocol becomes fully immutable, ensuring maximum decentralization. However, this also comes with a trade-off: greater decentralization reduces internal control, meaning there are fewer security backstops from the team.



Conclusion

After Hashlock's analysis, the FROGY App (dTelecom) project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.